目录

思考题参考答案 **1**

第一章 AI Agent 入门 1

第二章上下文工程 2

第三章用户记忆和知识库 3

第四章工具 5

第五章 Coding Agent 与通用 Agent 5

第六章交互：观察与动作空间的扩展 6

第七章 Agent 的评估 8

第八章模型后训练 9

第九章 Agent 的持续进化 11

第十章多 Agent 协作 12

思考题参考答案

本文件汇总全书十章思考题的参考答案提纲。思考题多为开放性问题，答案不唯一。参考答案由 AI 生成，经人工略作审校，仅供读者参考。建议读者使用 LLM 结合书稿内容进一步讨论这些问题。

第一章 AI Agent 入门

1. (★★) 如果你只能给一个 Agent 系统增加一项能力——更强的模型、更丰富的上下文、还是更多的工具——你会选哪个？在什么条件下你的选择会改变？

> 对应“大脑/眼睛/手脚”公式，先找短板：通常优先补上下文，即补充观察空间 (observation space)。若任务超出模型推理能力，换更强模型。若动作空间不足（例如无法访问公司内部系统），加工具。判断依据是分析失败轨迹，定位瓶颈在感知、决策还是行动。

2. (★★★) ReAct 循环中，累计缓存读取量随轮数近似二次方增长。如何降低这种增长？

> 第 i 轮读取的缓存前缀长度约与 i 成正比，累计读取量为 $1 + 2 + ... + n = O(n^2)$；这里二次增长的是累计缓存读取费用，而非轨迹长度或 KV Cache 占用。可在达到 token 阈值时批量压缩早期轨迹，只保留结论与关键状态，并把大块中间结果外置后按需检索，或用子 Agent 隔离。不能每轮压缩，否则既可能损害 Agent 性能，也会引入额外的压缩调用和缓存重建开销。

3. (★★) “模型即 Agent”范式意味着模型在工具调用决策上越来越自主。但本章论证了 Harness 工程的重要性反而在增加。这两个趋势如何共存？Agent 框架未来的核心价值体现在哪些方面？

> 马与缰绳隐喻：模型越强、自主空间越大，出错影响面越大，越需约束、验证、纠正。框架价值从“编排 LLM 调用”转向 Harness 五要素中的保障层：权限分类、熔断器、错误恢复、上下文压缩、工具生态。

4. (★★) 消融实验中“工具结果反馈”的缺失导致 Agent 陷入无限循环。在生产环境中，除了工具结果缺失，还有哪些情况可能导致 Agent 无限循环？你会设计怎样的检测和终止机制？

> 其他诱因：工具反复报同一错误、模型因幻觉调用不存在的工具、上下文压缩导致关键状态丢失、思考过程被剥离导致模型 API 报错、任务本身无解。机制：设定最大迭代次数等停止条件；检测重复调用（相同工具 + 参数指纹）；超过失败阈值后升级为人工干预。

5. (★) 本章用感知、行动、策略三个维度分析了五个 Agent 产品。请选择一个你日常使用的 AI 产品，用这三个维度进行分析，并思考它的架构设计是否合理。如果由你来设计这个 AI 产品，有哪些改进空间？

> 开放题。要点：仿照章中表格，写出眼睛（能看到什么信息源）、手脚（动作空间是否开放式、能否内部思考）、策略（Agent 执行循环的模式）。

6. (★★) 如果你要设计一个专门处理航班订票的客服系统，你会选择工作流模式还是自主 Agent 模式？有没有可能在同一个系统中混合使用两种模式？

> 主体采用工作流：身份核实 → 搜索 → 付款 → 预订四个节点，保证“付款前不能预订”等合规顺序，且把提示注入攻击面限制在单个节点内。开放性环节（理解需求、改签、航班取消后的替代方案推荐）切换为自主 Agent。高风险操作（大额付款、退款）需加人工确认。

7. (★★★) 护栏部分提到了工具风险评级。如果一个工具在大多数情况下是低风险的，但在特定参数组合下变为高风险（如 `delete_file` 删除普通文件 vs 删除系统文件），你会如何设计动态风险评估？

> 评级对象从“工具”细化到“工具 + 参数”：调用时根据可逆性、权限和影响面计算风险。采用基于规则的确定性检查（路径黑白名单、正则），而非模型判断。验证时应只检查结构化数据，防止提示注入操纵判断结果。

8. (★★) 本章的 Agent 产品表格中，所有 Agent 的动作空间都是“开放式”的。一个受限的动作空间（比如只能从预定义选项中选择）在什么场景下反而优于开放式？

高合规、高风险、错误不可逆场景：如退款、付款，受限选项即“约束”，天然防呆，从设计上让错误无法发生。

9. (★★) 人工干预机制要求 Agent 能“优雅地移交控制”。但在实践中，用户可能不在线、响应很慢、或者给出模糊的指令。此时 Agent 应该怎么办?

Fail-safe：高风险操作在未获确认时暂停，而非默认执行；先完成可逆的低风险部分，将高风险部分记录在文档中，便于人类决策和 Agent 恢复；使用异步沟通工具（消息、邮件）通知用户并设置超时策略；指令模糊时先澄清意图。

10. (★★★) 引言指出“好的设计原则应该穿越模型的迭代周期”，但实现这些原则的具体工程手段可能会随模型能力进步而过时。试举一个这样的 Agent 工程手段，并说明理由。

示例一：通过约束采样强制工具调用符合严格格式。这是在模型容易输出非法 JSON、遗漏参数时采用的可靠性补丁；随着模型的格式遵循能力提高，其收益可能逐渐降低，但高风险场景仍应保留确定性的格式校验。

示例二：为弥补模型无法持续吸收新知识而引入外部知识库。如果模型未来具备可靠的持续学习能力，一部分知识维护可能从模型外部迁移到参数中。不过，外部知识库在实时更新、精确检索、权限控制和来源追溯方面仍有独立价值，因此更可能缩小适用范围，而非完全消失。

示例三：要求所有能力都必须通过模型 API 的标准工具调用接口暴露，禁止自定义调用形式。Skills 已经展示了另一条路径：用文本描述能力及操作方法，再让模型通过通用命令行工具执行；从模型视角看，这相当于在通用执行器之上理解并遵循一种自定义的文本调用协议。随着模型理解任意接口的能力增强，“必须使用标准工具调用格式”不再适合作为普遍原则。标准格式对于互操作、结构化校验以及能力较弱的模型仍然有用，但它应是基于场景的工程选择。

示例四：要求提示词与全部工具定义必须预先放在上下文开头。这种做法源于早期模型的指令遵循能力有限，提示或工具定义离开熟悉的固定位置后，模型往往难以正确识别和执行。Skills 会在运行过程中按需把提示词加载到上下文中间；动态工具发现也会在找到新工具后，把工具定义追加到已有轨迹之后。随着模型的指令遵循能力增强，以及针对这类动态加载方式进行专门的后训练，提示词和工具定义不再必须固定在上下文开头。

第二章上下文工程

1. (★★★) 实验 2-3 发现，滑动窗口对话历史会导致 Agent 反复执行相同的工具调用。但完整保留历史又会让上下文不断膨胀。设计一种策略，既能避免信息丢失，又能控制上下文长度，且不破坏 KV Cache 前缀。

① 用压缩代替丢弃：消息只追加不删改，接近阈值（如窗口 80%）时批量压缩旧 tool results。② 分层机制：大输出落盘留摘要、噪声直删、归档式摘要保留脉络。③ 子 Agent 隔离，让中间状态不进主上下文。

2. (★★) Qwen3 的 Chat Template 思维链保留机制只保留“最后一个真实用户消息之后”的思考。如果一个 ReAct 循环跨越了上百轮工具调用，累积的思考内容可能消耗大量上下文。你会如何修改这个机制来应对超长循环? DeepSeek R1 曾要求剥离全部历史思考，而 DeepSeek V4 反转为强制回传全部 `reasoning_content`——对比这两种相反的策略，各有什么利弊? 这个反转说明了什么?

修改方向：滑窗保留——完整保留最近若干轮思考，在窗口外按 token 预算（而非固定轮数）触发滚动压缩，产出结构化状态栏（当前目标、已确认事实、已排除路径、待办）；压缩只发生一次且位置固定，缓存重建代价是一次性的，不必每轮承担。R1 剥离：节省 token、前缀稳定且有利于缓存，并与训练分布一致（历史 CoT 从不在输入中）；但每轮都要从零开始推理，会丢失长程计划，也容易重复犯错。V4 强制回传：思路连贯，长程 Agent 任务表现更好；但 token 成本高、每轮前缀都会膨胀，且无法从非思考模式无缝切换。这个反转说明：对纯对话场景而言，思考是废料；对 Agent 任务而言，思考则是状态——行业实践已转向后者。

3. (★★) 上下文感知压缩实验中，从约 148K 个字符压缩到约 2,000 个字符，这种极端的压缩是否存在“不可逆信息损失”的风险? 如何解决?

有风险，压缩是一种有损投影；如果问题涉及未保留的信息，就无法得到可靠答案。解法是采用“有损压缩 + 无损索引”：每条事实都附上来源 URL，以便回溯；原始输出存入磁盘，平时只查看摘要预览；明确保留信息的优先级——架构决策、保证语义完整性所需的信息（时间、公司名）、验证状态以及 UUID/hash 等标识符均原样保留；通过自适应窗口化推迟压缩时机。

4. (★★) Agent 状态栏将隐式状态显式化。但如果状态栏本身包含了错误信息（比如工具计数器出了 bug），Agent 可能基于错

误的信息做出有害的决策。这种“元信息可靠性”问题如何缓解?

> 模型几乎会无条件相信状态栏，其中的错误也会原样传导。缓解措施：① 用确定性代码维护，绝不让 LLM 批量统计很长的历史记录(即便使用 LLM，也只让它逐条抽取，再由代码汇总)；② 把状态栏准确率作为一线生产指标持续监控；③ 只采用来自真实世界可靠观测的信息，防止状态栏遭到投毒。

5.（★★）提示工程消融实验表明，信息组织的混乱导致成功率下降 30% 以上。但在实际开发中，系统提示词往往由多人在不同时间维护。你会用什么工程实践来防止系统提示词的“熵增”?

> ① 把提示词当代码：版本控制、评审，产品经理定业务规则、工程师负责编码；② 用 Tau-Bench 类基准测试做回归测试，改动前后跑消融实验定位影响；③ 强制结构化：SOP 流程驱动而非规则堆砌，XML/Markdown 分层；④ 片段按“可缓存/破坏缓存”分类命名，动态内容归到缓存边界后；⑤ 膨胀内容拆成 Skills 按需加载。

6.（★★★）本章提出“上下文学习本质上是检索而非推理”。如果这个论断成立，当前所有基于“把更多信息塞进上下文”的优化方向都需要重新审视。你认为应该如何突破这一局限?

> 为“只有一半的检索引擎”补上提炼层：① 采用上下文蒸馏或状态栏，用代码提前算好结论，供模型直接检索；② 主动压缩，把原始记录转化为高密度的结构化知识；③ 用子 Agent 隔离，避免噪声进入主上下文；④ 把交互作为第三条路径，将外部仪器观测到、模型无法自行推导的新信息写回上下文；⑤ 探索可编辑、可组合的 KV Cache“笔记”和跨会话记忆沉淀。

7.（★★★）Skills 的渐进式披露只在 Agent 判断需要时才加载完整内容。但这个判断本身依赖模型的能力——如果模型不知道自己不知道什么，就无法正确触发 Skill 的加载。这个“元认知”问题如何解决?

> ①Skill 的元数据（名字、描述）常驻上下文，让模型始终“知道自己拥有什么”；②Skill 的 description 写成路由条件而非功能介绍：“Use when / Don’ t use when”，避免宽泛描述。

8.（★★）Skills 机制中，Agent 从 SKILL 文件中动态读取提示词之后，后续的操作能否正确遵从这些指令?不同的模型对 Skills 模式的支持有什么区别?

> 取决于 Skill 的注入方式：注入 system prompt 遵循最强但破坏 KV Cache；作为普通文件读到上下文中间，模型的指令遵循可能较差；注入到上下文末尾，指令遵循较好，但每次工具调用都需要重新计算 skill 部分的 KV，成本较高。

9.（★★★）本章强调动态信息（如系统时间戳、工具列表顺序）的变化会破坏 KV Cache 前缀命中。在一个拥有大量工具且工具集频繁变动的生产系统中，你会如何设计上下文布局来最大化缓存命中率?

> ① 少量稳定核心工具（如七个）+通用执行器，具体能力走 Skills 渐进披露，工具定义冻结在静态前缀、固定顺序；② 子 Agent 与父 Agent 前缀保持相同。

第三章用户记忆和知识库

1.（★★）在用户记忆系统中，当同一用户在不同会话中提供了矛盾信息（比如两次提到不同的家庭住址），记忆系统应该如何处理这种冲突?

> 采用 Mem0 式“提取—对比—决策”流水线：先通过向量检索找出相近的旧记忆，再由 LLM 判定 ADD/UPDATE/DELETE/NOOP，如“搬到上海”应通过 UPDATE 覆盖“住在北京”；同时进行版本化管理，地址类信息只保留最新版并标记时间戳，工作经历类信息则保留完整历史；检索时可借助上下文前缀（人物、时间、意图，如电汇信息三次修改的案例）判断哪条信息最终有效。

2.（★★）上下文感知检索将原始文档的上下文附加到每个分块。但如果原始文档本身结构混乱或存在矛盾信息，这种方法可能传播甚至放大错误。你会如何在检索阶段引入“信息质量”信号?

> 可以借鉴“知识库时效与治理”的思路：给分块附加版本号、生效/失效时间、来源等元数据，检索时过滤已失效内容，或在前缀中显式标注“此条已于某日废止”；重排序阶段把来源权威性和时效性纳入评分，而非只看语义相关性；在索引阶段，让生成前缀的 LLM 同时检测并标记分块间的矛盾，类似记忆系统中的版本化冲突检测。

3.（★★）第四章介绍的多模态信息提取，会先把图表转为文本描述再进行检索。这个“翻译”过程可能丢失视觉信息中的空间关系。

举一个具体例子，说明纯文本描述无法完整传达的图表信息，并设计一种保留该信息的方案。

> 例：系统架构图中的逻辑关系，折线图中两条曲线的交叉点位置，或 PDF 表格中单元格与表头的行列对应。方案一：原生多模态处理；方案二：提供多模态图片分析工具。

4. (★★★) Rich Sutton 的“苦涩的教训”认为通用方法（搜索和学习）最终会胜过手工设计的特征。本章构建的整个知识系统（分块策略、索引结构、检索管道）是否本身就是一种“手工设计”？如果模型能力足够强，这些设计是否会被简单的“全量输入”所替代？

> 确实是手工设计，部分环节（分块、融合调参）可能随长上下文而弱化；但黑猫白猫案例表明“全量输入”也不够：注意力是软检索，跨文档聚合统计仍需索引期预提炼；知识过期更新、权限/租户隔离、可审查性、成本这些工程约束与模型能力无关；且检索与索引期 LLM 提炼本身就是“搜索 + 学习”的通用方法，并非与苦涩教训对立。

5. (★★★) 随着模型能力的提升，你认为领域知识库还重要吗？未来强大的基座模型是否有可能包含领域知识库中所有的信息，从而不再需要领域知识库？

> 仍然重要：训练数据有截止日期，知识库可随时更新；企业内部流程、私有判例等根本不在公开语料中；多用户共享需要进行权限过滤与租户隔离，参数中的知识无法按调用者裁剪；外部存储可审查、可进行版本控制，也可以使失效内容下线，这些都很难通过参数记忆做到；即使采用参数化路线（后训练 / User as Engram），也面临“记住容易，用来做多跳推理却很难”的问题。

6. (★) RAPTOR 通过自底向上的层次摘要构建树形索引，GraphRAG 通过实体关系构建图结构索引。这两种结构化索引分别擅长回答什么类型的查询？

> RAPTOR：从宏观概念逐步钻取细节的“跨层穿梭”式查询，如先定位“SIMD 指令集”摘要再下钻到 SSE 细节，兼顾总览与细节两种粒度。GraphRAG：多跳关系推理（“我的医生所在医院的地址”沿关系链遍历）与实体消歧（两个“张医生”是不同节点）等“A 和 B 有什么关系”类查询，社区摘要还提供主题聚类。

7. (★★) 文件系统范式将知识组织为类似文件系统的层次结构。这种方式和传统的向量数据库 RAG 相比，在什么场景下更有优势？

> 纯文本可被用户直接阅读、编辑、修正，可用 Git 版本控制与回滚，适合需要人机共同维护、审查知识的场景；Agent 有 write_file 能力即可自主记录经验，形成记忆自进化循环（外部化学习）；L0/L1/L2 渐进披露使多数查询到 L1 即可决策，省 token；前提是像 Wikipedia 一样建立交叉链接和索引页，否则孤立文件越多越难检索。

8. (★★★) 从结构化数据（如司法判决数据库）中自动发现“裁判因素”和“因素重要性层级”，本质上是让 Agent 从数据中归纳规则。这种数据驱动的知识提取是否能达到人类专家手工编写规则的质量？

> 优势：如 CAIL2018 实验所示，“自下而上”的因子发现更贴合数据，而非人类先验，能捕捉散落在成千上万份判例中、专家难以显式写出的隐性权衡经验，而且可以量化。局限：LLM 提取出错会造成知识污染，数据本身的偏差也会被继承，聚类原型只能反映相关性，无法说明因果关系。折中方案：采用数据驱动建模，由专家审核 Schema 与结果；由模型提出问题，用统计结果支撑解释。

9. (★★★) 请为一个 Markdown 用户记忆库同时设计增量更新与定期整理流程。如果 Reviewer 与 Proposer 使用同一模型，且只能看到 Proposer 挑选的对话片段，系统仍可能合入哪些错误？请从模型独立性、证据覆盖和工具权限三方面说明你的改进。

> 增量更新：把记忆库当代码库，每条变更走一次 PR。Proposer 先检索相关旧知识，再提出尽可能小而完整的 diff，同步维护链接、索引、时间元数据与证据引用；Reviewer 拿变更前的知识、diff 和原始证据独立审核，退回时给出指向具体证据和行号的可执行意见；迭代设最大次数或成本上限，超限转人工而不是默认放行。合入后先由 CI 检查格式、链接、元数据与权限标签，再从已合入版本增量重建受影响的分块与向量索引。定期整理：按时间或按新增条目数触发全量扫描，去重、合并、拆分过大文件并重建入口页；关键是回到原始对话逐段核对，检查旧摘要有没有丢掉否定词、时间条件或限定语；遇到互相矛盾的说法不按“保留最新”收敛，而是追溯各自来源、写清适用场景，证据不足时保留冲突与待确认状态。重组同样以 PR 提交，可按目录拆成多个 PR，全部通过后除重建索引外还要回放一组典型检索用例，确认原本能找到的知识没有变得不可见。
>
> 使用相同模型并仅输入部分片段，会漏掉三类错误。**模型独立性**：同源模型共享训练先验与盲点，Reviewer 容易顺着 Proposer 的结论复述，而不回到证据，同一处误读因此不会被发现——应改用能力相近但来自不同家族的模型互审。**证据覆盖**：只看 Proposer 挑选的片段，断章取义、被丢弃的否定词与前置条件以及与其他文件的冲突都无从暴露——Reviewer 必须能在其被授权的租户或用户范围内，自主检索完整的知识库与原始证据库，而不是只接收上游选好的几段。**工具权限**：若 Proposer 能直接写入主分支或修改

> 线上索引，审核就形同虚设——应强制分工，Proposer 只写工作分支，Reviewer 只读证据并提交审核结论，只有合并流程可以更新主分支与线上索引，验证器与发布门槛本身不在可修改范围内。

第四章工具

1.（★★）MCP 标准将工具定义从 Agent 框架中解耦了出来。但标准化也意味着复杂的工具交互模式（如流式输出、双向通信、有状态会话）可能难以在标准协议中表达。你认为 MCP 未来最需要扩展的能力是什么？

> 最需要扩展的是跨会话的事件驱动能力。MCP 已经能够支持多轮交互、变化订阅和长任务，但它的核心仍是对单次能力调用进行标准化，而不是让 Agent 持续在线。新邮件、外部回调等事件如何唤醒 Agent，多个事件如何排队、恢复和重试，仍需要 Agent 框架自行处理。未来如果能在不破坏工具协议简洁性的前提下，为这类事件编排提供更统一的约定，MCP 的适用范围会进一步扩大。

2.（★★）在 MCP 生态中，不同的 MCP 服务器可能提供功能高度重叠的工具。当 Agent 面对多个来源不同但功能相似的工具时，应该如何选择？如果不同来源的同名工具在行为上略有差异（比如一个返回摘要，另一个返回全文），Agent 是否有能力感知并利用这种差异？

> 选择依据：接入前审查描述、锁定版本、配最小权限凭证，警惕同名工具遮蔽（tool shadowing）把敏感调用路由给恶意方；运行时靠层次化分类和动态发现缩小候选。模型是否能感知差异，取决于工具描述的质量。

3.（★★）本章提出了“执行-验证-反馈”闭环（如写代码后自动运行 linter）。这种“操作后立即自动验证”的模式还可以应用到哪些工具场景？是否存在某些操作，其验证本身的成本或风险超过了操作本身，导致这种模式不可行？

> 可泛化的场景：修改配置后在沙盒中实际运行，验证配置是否生效；生成文档或演示文稿后渲染成截图，利用模型的多模态能力检查排版。不适用的场景：发邮件、拨电话、对外转账等不可逆或非幂等操作。此类操作要么无从验证，要么验证本身会再次触发真实世界事件；此时应改用事前控制手段，如提议者-审核者的事前审批。

4.（★★）本章提出了“工具爆炸”问题——Agent 面对数千个工具时选择精度下降。除了主动工具发现，还有哪些方案？可以参考人类专家在面对大量可用工具时的策略。

> ① 层次化分组：先定位“服务器/App”再选具体工具；②Skills 式“按需查阅”：像查工具书，目录常驻上下文、细节按需加载；③ 少数常用基础工具“放在手边”常驻上下文，其余靠目录索引。

第五章 Coding Agent 与通用 Agent

1.（★★）代码生成被称为 Agent 的“元能力”。但代码执行引入了安全风险——Agent 生成的代码可能包含漏洞、无限循环或资源耗尽。沙盒隔离能解决部分问题，但也限制了代码能力（比如无法访问网络或文件系统）。如何在安全性和能力之间找到最优平衡点？

> 沙盒按场景分级隔离（容器/microVM）；网络默认断网、白名单代理按需放行；源码只读挂载、API key 不要放在沙盒内；沙盒资源限额；沙盒生命周期管理（超时）。

2.（★★★）Agent 自举——能创造 Agent 的 Agent——实现了“智能的自我繁殖”。但每次自举都可能引入新的偏差或错误，这种错误会在代际间累积吗？如何防止 Agent 自举的退化？

> 若每一代都在上一代产物的基础上继续繁殖，一些缺陷可能会累积。关键是要有足够有挑战性的 verifiable task（可验证任务），例如足够困难的编程任务。

3.（★★）代码生成 Agent 在处理日志解析时，能自动跟随格式演化。但如果格式变化是一个 bug 而非预期改动，Agent 的适应性反而掩盖了问题。Agent 应该如何区分“需要适应的变化”和“需要报告的异常”？

> 适应前先诊断：对照架构文档与 PRD 判断新格式是否符合预期（实验 5-11 的思路）；核对版本控制记录，确认变化源自合法的代码提交，还是没有明确来源的漂移；类比 τ-bench 的 log_mismatch，即使选择适应，也要记录告警、自动创建 issue，而非静默兼容；不确定时引入人在回路进行确认。原则：适应与报告并行，不能让适应机制吞掉异常信号。

4.（★★）本章在 PPT 生成、视频编辑和日志可视化中反复使用提议者-审核者机制。如果 Reviewer 的审美偏好与目标用户不一

致，比如 Reviewer 认为信息密度合理但用户觉得太拥挤，反馈循环会收敛到错误的局部最优。如何让用户的偏好反馈也参与 Reviewer 循环？

> 把用户反馈作为最高优先级的结构化事件注入 Agent 轨迹；将用户偏好记录在外部的 MEMORY.md 中，使其能够跨任务生效；交付 HTML 格式的文档而非 Markdown 文档，以便用户查验。

5.（★★）本章展示了 Coding Agent 把执行和调试中获得的经验沉淀回代码库的多种方式——写入知识库文件、更新架构文档、维护项目指令文件、把操作序列固化为代码。如果把这些经验进一步提炼为系统提示词中的规则，规则集会随时间不断膨胀。如何对沉淀下来的规则做“垃圾回收”——识别并清理冗余或过时的条目？为什么一次成功的代码修改还不能直接视为第九章所说的持续进化？

> GC 思路：能编码进 Linter、CI 或工具校验的规则移出提示词；追踪规则命中率和冲突，定期对照代码库重新验证；用 Markdown 与 Git 保留来源、版本和回滚能力。一次补丁成功只说明它解决了当前案例；持续进化还要求修改来自可追溯的运行证据，能改善后续任务，并通过旧任务回归与安全验证。

6.（★）“对远程工作友好的团队往往也对 AI Agent 友好。”你所在的团队或组织，在知识文档化方面距离“AI-ready”还有多远？最大的障碍是什么？

> 开放题。可用本章的代理指标自查：远程新人只靠仓库和文档能否独立开展工作。检查项：决策是否记录在文档中、上下文是否写进 issue/PR、构建和测试命令是否记录在 CLAUDE.md/AGENTS.md 一类的指令文件中、部落知识是否沉淀为开发者指南。最常见的障碍是依赖“问旁边同事”的口头传递与白板文化——Agent 读不到口头约定，只能读到文档。

7.（★★★）Simon Willison 提出了 Agent 的“致命三要素”（访问私有数据、暴露于不受信任内容、具备外部通信能力），本章在此基础上增加了第四个——持久记忆。在一个需要同时处理这四种要素的生产环境中，你会如何设计安全策略？

> 按四类边界分层设防。数据边界：不挂载凭证，以只读方式挂载源码，尽量缩小可见范围。输入信任边界：标注来源，将外部内容降格为“可参考、无指令效力”的数据（忠诚度守则）。输出影响边界：默认断网并设置白名单出口、解析命令语义而非采用黑名单、使用 Sidecar 独立复核并引入人在回路——关键操作必须由上下文之外的机制复核。跨会话边界：写入 MEMORY.md 的内容需经过与外部内容同等严格的信任审查。目标是即使受到提示注入，恶意指令也无法执行。

8.（★★）Artifact 模式让 Agent 生成 SQL 或前端代码，由数据库和浏览器直接执行，绕过 LLM 处理大量数据。与传统的“Agent 直接给出答案”相比，这种“Agent 生成代码、系统执行代码”的分工有什么优劣？生成的 SQL 可能执行破坏性操作、生成的 HTML 可能包含漏洞，又该如何确保安全？

> 优劣：数据从数据库直达前端，绕过 LLM 这个“中间人”——速度快、节省 token，还能避免抄写大量数据时产生幻觉错误，适合呈现大量数据；代码可审计、可复用，还能组成流水线（SQL 结果直接传给可视化代码）。代价是 LLM 看不到查询结果，无法基于数据内容进一步归纳和决策，不适合需要模型先消化数据再推理的任务。
> 安全：SQL 用最小权限只读账号执行，并添加 CPU、内存等资源限制，防止资源耗尽；HTML/UI 优先 A2UI 类声明式协议，Agent 只输出界面描述 JSON，由客户端用受信组件目录渲染，不执行任意代码；确需任意 HTML 时应在沙盒环境中展示，防止注入。

9.（★★）将业务规则编码为工具内部基于数据库真值的校验，并用参数设计引导模型在调用前核对政策条件，本质上是用代码结构来约束 Agent 行为。这种“代码即规则”的模式相比自然语言规则有什么优势和局限？

> 优势：没有歧义、能够确定性执行、擅长处理复杂的条件组合；政策事实取自数据库真值和服务端时钟，不采信模型自报值，幻觉和提示注入都无法绕过，是防止不可逆操作的最后一道防线；expected_* 参数还可兼作强制 checklist，引导模型思考。局限：代码不会向用户解释政策，也不会寻找变通方案，而且有维护成本。结论：代码规则与自然语言规则互补，而非相互替代。

第六章交互：观察与动作空间的扩展

1.（★★）在异步 Agent 架构中，事件队列的优先级策略需要在设计时确定。但如果优先级判断本身需要语义理解（比如判断一条新消息是否比当前任务更紧急），这个判断应该由谁来做——规则引擎还是另一个 LLM 调用？各有什么代价？

采用分层混合方式：类型明确的事件用规则硬编码，零延迟、确定性强，但无法理解“马上停下来”与“今天天气怎样”的语义差异；语义模糊的事件交给轻量级分类 LLM，由其充当事件路由器，代价是增加数百毫秒延迟和额外费用，而且可能误判；同时需要像 Sidecar 一样只读取结构化字段，以防范提示注入。

2.（★★）在队列式事件处理中，模型倾向于只关注最后一个事件，本章通过 Agent 状态栏标记和汇总来缓解。但如果队列中积压了 20 个事件（10 个工具结果 + 5 条用户消息 + 5 个系统提醒），你会如何组织这些事件的呈现顺序和格式，使模型不遗漏关键信息？

先用规则和轻量级 LLM 分类去重：紧急事件（告警、用户中断）单独采用取消式处理，不混入批量事件。将 10 个超长的工具结果截断后持久化到文件，只保留开头、结尾和文件路径。在上下文末尾的系统状态栏中加入汇总清单（各类事件的数量，以及逐条回应的要求）。

3.（★★★）Agent 代表用户与外部世界交互时，本质上面临一个身份选择：是用独立的虚拟身份（专属邮箱和电话号码）以第三方身份行动，还是直接以用户本人的身份操作其个人账号？前者可以在后台自主操作，但第三方可能不信任一个非真人的身份；后者拥有更完整的上下文和权限，但引入了信任授权和安全边界的问题。你认为在什么场景下应该选择哪种模式？

默认使用虚拟身份：可以在后台自主操作且便于审计，出错或被攻破时也不会暴露用户的全部数字身份，就像秘书使用自己的办公邮箱；但需要应对 CAPTCHA/IP 信誉问题（住宅代理）。在必须以本人身份操作的场景中（账户身份验证、三方通话确认，如 Pine 打客服电话），采用 Human-in-the-loop 认证：通过 VNC/RDP 让用户在可视化界面中亲自登录。判断标准包括对方是否要求账户持有人本人操作、操作风险和凭证范围。

4.（★★）语音 Agent 的端到端模型将 ASR-LLM-TTS 合并为单一模型，降低了延迟却失去了模块化。如果端到端模型在某个环节（如语音识别）出错，调试和修复比串行管道困难得多。你会如何设计端到端语音 Agent 的可观测性（observability）系统？

让模型同时输出可读的中间表示，如 Moshi 的“内心独白”文本流和声学事件标记（`<emotion>`、`<noise>`）；用“自级联”定位出错环节：让同一模型先转录再推理，并与端到端结果对照，判断错误出在感知还是思考；离线按照副语言理解、轮次判断等维度进行分项回归测试。

5.（★）Step-Audio R1 通过 MPS 双脑架构实现“边想边说”。但人类在“边想边说”时经常会说出未经深思熟虑的话、自我纠正、或使用填充词。Agent 的“边想边说”应该模仿人类的这些特征吗？

应该模仿具有信号价值的“不完美”：停顿、填充词是思考的外化，能够掩盖延迟，其插入位置可由 LLM 决定；不应模仿会破坏信任的自我纠正：方案一中的快慢思考矛盾（“到底买不买?!”）会让信任崩塌；MPS 实验显示 CoT 开头多是在复述问题，因此提前开口说些铺垫是安全的，无需先说错再改口。

6.（★★）SoM（Set-of-Mark）及其结构化变体（DOM 元素索引）将 Computer Use 的视觉定位从开放坐标预测转为封闭 ID 选择，但都需要先检测和标注界面元素——无论靠分割模型还是靠 DOM。如果界面包含非标准控件或动态变化的元素，标注就可能不完整或不准确。这种情况下应该回退到坐标预测吗？

应保留坐标预测兜底：它是唯一不依赖标注的路线，非标准控件、动态元素均适用；更实用的是混合 action space：标注可得的元素仍用 ID 选择。坐标预测须做分辨率匹配与等比缩放，否则会发生系统性偏移。

7.（★★）XLeRobot 等几百美元级机器人平台让遥操作数据收集变得廉价。但遥操作数据的质量高度依赖操作者的技能。一个不熟练的操作者提供的数据会如何影响 VLA 模型的训练？如何在数据收集阶段自动筛选低质量数据？

VLA 主要靠模仿学习，低质演示会把抖动、绕路、犹豫与失败动作当成正确策略学进去。呼应第八章的判断：数据比架构更关键。

8.（★★★）本章覆盖了语音、Computer Use 和机器人三种交互形态。交互架构可以通过端到端统一、模块化级联或前台交互与后台推理解耦来改进，而不必和智能上限沿同一条路线增长。未来五年的 Agent 应该优先追求更强的统一模型，还是保留可替换的快慢分工？请结合延迟、可观测性、模型迭代速度和任务风险讨论。

按 Thinking Machines Lab 主张，交互性将内建于模型而非外挂 harness，随智能一同扩展；Computer Use 从逐帧截图走向连续观察；具身智能的世界模型将全面实现，但由于前沿推理模型发展很快，快慢解耦不会消失，交互模型与 SOTA 思考模型的快慢思考协同架构可能成为长期架构。

9.（★★）DOM/Accessibility Tree 元素索引在标准 Web 应用上效果显著，但越来越多的软件界面（Canvas/WebGL 渲染、跨平台自绘控件）不提供可访问的结构化信息，只能依靠视觉标注或坐标预测。你认为 Computer Use 应该押注纯视觉路线，还是

同时维护结构化和视觉两条路径？维护两条路径的成本和收益分别是什么？

> 短期内两条路径并存：能够取得结构化索引时，定位最准确、最稳定，还能避免分割模型误检；纯视觉则是原生软件、Canvas 和游戏中的唯一选择。当模型本身的 grounding（点击指定坐标）能力较强时，使用结构化索引方案并不会体现出显著优势。长期来看，纯视觉路线的上限更高。

10.（★★）VLA 模型采用动作分块（action chunking）——如正文所述，模型一次生成一小段未来动作，由控制线程以更高频率回放——将推理延迟隐藏在执行时间里。但如果执行过程中环境突变（如物体被移走），预生成的动作序列就会失效。如何在动作分块的效率优势和环境变化的响应速度之间取得平衡？

> 分块本质上是用反应速度换取动作的平滑性，块越长，反应越迟钝；块长只需满足“推理时间 < 块执行时间”的下限，不应盲目加长；执行过程中让感知模型持续运行，检测到环境突变就丢弃剩余动作并重新推理，相当于语音场景中的“打断”。可根据场景动态调整块长：静态场景使用长块以节省算力，动态场景使用短块以保证响应速度。

11.（★★★）本章的三个场景（语音、Computer Use、机器人）都面临“感知-思考-行动”循环的延迟问题，都需要在智能上限和交互时效之间做分工。在语音场景中，这表现为“说错了再纠正”；在 Computer Use 场景中，这表现为“先点再看”；在机器人场景中，这表现为“走一步看一步”。如何通过动作分级、可逆操作、状态确认、权限控制和安全停止，保证快速交互不会导致无法挽回的后果？

> 按可逆性给动作分级：快思考只允许执行可逆动作，不可逆操作则交由慢思考把关；不允许快模型执行会造成不可逆后果的工具调用。

12.（★★★）本章反复出现同一组原语（唤醒、安全点、取消、抢占、快慢分离）在不同时间尺度上的实现。请任选其中一个，说明它在事件驱动（秒—天）与机器人动作分块（毫秒）两处的实现差异；这种差异主要由什么决定——环境变化的速度、动作的可逆性，还是观察的获取成本？

> 以“取消”为例。事件驱动中的取消发生在两次工具调用之间的安全点：Agent 收到 terminate 后清理资源、返回确认再退出，延迟以秒计也可以接受，因为一次工具调用本身就要几秒到几分钟。动作分块中的取消则必须在毫秒内生效：控制线程一旦发现安全事件或观察显著变化，就要立刻停止当前动作、丢弃剩余 chunk 并重新观察，慢一步就可能撞上障碍物。
>
> 在三个候选因素中，**观察的获取成本**其实最不关键——两边都能以较低成本重新观察。真正决定差异的是另外两个因素的组合：**环境变化的速度**决定安全点必须设置得多密集（工具调用之间设置即可，还是每个控制周期都要设置），**动作的可逆性**决定错过安全点的代价（多发一封邮件还可以再补一封道歉，撞倒的杯子却无法复原）。
>
> 由此可以提炼出一条设计规则：安全点的密度应与环境变化速度匹配；而在安全点之外还需要多少额外防护（硬件急停、独立安全控制器、二次确认），取决于动作有多不可逆。这也解释了为什么第四章的高风险操作要采用事前审批，而机器人要使用独立于模型的硬件安全层——两者都是在“安全点不够用”的地方补上的第二道防线。

第七章 Agent 的评估

1.（★★）LLM-as-a-Judge 使用语言模型评估语言模型的输出。这种“自我评估”是否存在系统性盲区——比如模型可能一致地给某种风格的回答打高分，而这种偏好与人类评判不一致？如何检测和校正这种偏差？

> 存在：长度偏差、回答风格偏差、同源模型被钻空子（古德哈特定律）。检测：建 100-200 例人工金标集，测评判与人类的 Cohen′ s kappa；定期审计评分与回答长度的相关性；红队构造对抗案例。校正：Rubric 显式惩罚冗长、限长度；不同模型家族多源异构评判。

2.（★★★）评估数据集的“防泄漏”设计至关重要。但在开源生态中，benchmark 数据一旦公开，很快就会被纳入训练数据。这场“猫鼠游戏”有终局吗？设计一种从根本上抵抗数据泄漏的评估方法。

> 静态题库无终局，只能追赶。根本出路是公开“生成机制”、私有化“具体实例”：像 τ^2-bench、AndroidWorld 那样参数化模板每次随机实例化，验证基于最终环境状态而非固定答案序列。

3.（★★）Scale AI 的四准则（基于专家指导、全面覆盖、按标准重要性加权、评价标准自包含）旨在消除评估的主观性。但某些任务维度（如“回答是否有帮助”“语气是否恰当”）天然具有主观性。如何为这些主观维度设计可靠的 Rubric？

> 把抽象标准转化为可验证的行为。每一档都配上具体示例和边界案例；Rubric 是迭代的产物——在试用中收集评价者的分歧，逐渐演化为判例集。再辅以多个评委的加权评分和一致性检查，将存在分歧的案例送交人工复核，并在金标集上校准一致率。

4.（★★）τ-bench 通过模拟真实用户行为来评估 Agent。但模拟用户本身也是一个 LLM——它可能系统性地低估某些边缘场景

(如情绪激动、表达不清的用户)。如何验证模拟用户本身的质量?

> τ-bench 初版的教训是:模拟器过于机械、指令过于简单(Agent 能猜对答案)。验证手段:人工抽检模拟对话,检查模拟器是否遵守渐进式披露原则、是否编造脚本之外的信息;用小样本真实用户进行测试,观察其模型排名是否与模拟评估一致。

5. (★★) 配对比较(Bradley-Terry 模型)假设偏好是传递的(如果 A > B 且 B > C,则 A > C)。但人类偏好经常违反传递性。在 Agent 评估中,非传递偏好可能出现在哪些场景?这如何影响排名的可靠性?

> 场景:多维权衡时(A 准确但慢、B 快但简略、C 详尽但贵),不同评判者/任务看重的维度不同。Chatbot Arena 的排名本就依赖用户提问分布。影响:BT 把实力压成单一分数,非传递时排名不稳定、随对局分布漂移。缓解:按能力维度分别排名、报告两两胜率矩阵。

6. (★★) 本章区分了 Pass@k 的能力上限与 Pass consecutive@k 的业务可靠性。对于一个单次成功率只有 60% 的 Agent,怎样结合任务的失败成本、重试成本和副作用,决定应该报告哪个指标、取多大的 k?

> 先看失败是否可回滚。失败能自动重试、又不产生外部副作用时(检索、草稿生成、代码补全),关心的是"给足机会能不能做成",应报告 Pass@k,k 取实际允许的重试次数;失败会留下不可逆后果时(付款、退款、对外发信、生产部署),一次错就是真实损失,应报告 Pass^k。单次成功率 0.6 时,Pass@5 ≈ 99.0%、Pass^5 ≈ 7.8%——同一个 Agent 的两个数字相差一个数量级,只报前者会严重高估可靠性。
>
> k 的取值应来自部署现实,而不是好看的数字:Pass@k 的 k 取重试预算,Pass^k 的 k 取一次值班或一批任务中连续执行的次数。重试成本高时可以两段式验收——先用 Pass@1 粗筛方案,再对少数候选跑 Pass^k。无论报哪个指标,都必须写清 k 和采样口径;对有副作用的操作,应在沙盒或可回滚环境中采样,并把每一次失败都计入可靠性统计,而不是"重试到成功为止"。

7. (★★) 本章提出"观察 → 假设 → 实验 → 验证"的科学方法。但在实践中,Agent 的行为空间巨大,验证一个假设可能需要数百次评估运行。如何在有限计算预算下最大化评估的信息量?

> 先用失败聚类把范围缩到最有信息量的任务,再做低成本、单变量的配对试验,把小样本当作扩大测试的门槛,而不是部署证据。统计上,先用标准误做保守筛子;同批任务用 McNemar 一类的配对分析;预期分差小于噪声带宽时应扩充评估集。若并行筛选多个方案,还要校正多重比较,并对正向结果做独立复跑。

8. (★) AndroidWorld 小实验中,完整元素树把成功率从 25% 提高到 100%,却把 token 用量推到 2.498 倍;精简后成功率不变,token 降到对照组的 0.506 倍。怎样设计一套自动裁剪规则,既删掉无语义的 UI 节点,又不误删对可访问性、状态验证或后续操作有用的信息?

> 可采用"默认删除、证据保留"的分层规则:保留可见、有文本、可操作、可聚焦、可滚动、带状态值或无障碍标签的节点,同时保留这些节点到根的最短祖先链和必要的相邻标签;删除没有语义的布局容器,并对重复子树做摘要。裁剪前后要校验可操作元素 ID、状态和值是否守恒,还应保留截图作为视觉兜底。规则先在失败轨迹上回放,再用未参与调参的应用做回归;成功率、token 和延迟都作为护栏,任何可访问性任务退化都应阻止发布。

9. (★★) τ-bench 的用户模拟采用了"渐进式信息透露"——不一次性提供所有信息,而是根据 Agent 的提问逐步透露。这种设计如何影响评估结果?如果模拟用户的信息透露策略与真实用户差异较大,评估结论还可靠吗?

> 影响:若透露策略失真,Agent 可能只是学会了"适配模拟器"(古德哈特),绝对分数不具备参考价值;模型之间的相对排序可能仍有参考价值。补救:用真实对话校准模拟器、人工抽检、明示结论适用边界。

第八章模型后训练

1. (★★) 灾难性遗忘——一次针对特定任务的微调破坏了模型原有的通用能力(如通用工具调用)——在 Agent 场景下尤其棘手。相比全参微调,LoRA 冻结基座权重、遗忘风险更低,但并非免疫。有哪些策略可以进一步缓解微调带来的能力遗忘?

> 数据配比:混入约 20% 的通用或原分布数据,避免新任务数据占比过高而破坏旧能力;控制训练量:SFT 达到"格式稳定、能力初具"时即停止,通过早停防止塌缩;RL 采用较小的 rank (8–32) 并保留 KL 惩罚,使策略保持在参考模型附近;冻结关键组件(如 VLM 只训练投影层);按任务挂载多个 LoRA adapter 以隔离能力;用通用基准进行回归测试。

2. (★★) 后训练将能力固化为模型权重（“肌肉记忆”），而上下文学习将知识放在推理时的输入中。但有些能力（如领域知识）既可以通过后训练学习，也可以通过 few-shot 示例提供。你会用什么标准来决定某项能力应该走哪条路径?

> 首先看能力能否被外部符号充分表达：事实与证据适合 RAG，可语言化原则适合 Prompt/Skill，确定性流程与硬约束适合程序；医疗影像理解、自然语气和隐式策略等高维能力即使领域仍在变化，也往往需要参数更新。再看更新成本、调用规模、时效和风险：探索期先用上下文快速验证，稳定有效且需要广泛泛化时再训练；硬性规则无论多稳定都不应只依赖参数记忆。

3. (★★) 模型蒸馏让小模型学习大模型的行为。按能力层次，被蒸馏的模型大致可分为三级——Chat 模型**（单轮对话、直接作答）**、Reasoning 模型**（带长链思考再作答）**、Agentic 模型**（多轮调用工具、与环境交互）。分别蒸馏这三类模型，难点有什么不同？（提示：从“要蒸馏的到底是什么”入手——是输出的风格、完整的思考轨迹，还是与环境交互的决策策略；轨迹里哪些 token 该学、哪些是环境返回的不该学；以及成败信号出现得有多晚、有多稀疏。）**

> Chat: 只需学习 “输入 → 输出” 的映射与风格，采用标准 SFT 即可，最为简单。Reasoning: 需要学习完整的思考轨迹，因此要使用开源教师模型；同时必须过滤答案错误的轨迹。Agentic: 需要真实的仿真环境；离线学习容易出现 learner-sampler mismatch，建议基于开源教师模型进行 On-Policy Distillation。

4. (★★★) 在多轮 Agent 交互中，奖励的归因（credit assignment）问题比单轮更严重——一个最终的成功或失败很难归因到第 3 轮还是第 7 轮的决策。你会如何设计奖励分配策略?

> 中间步骤可以判定时，加入过程奖励（V-IRL 每步 ±1）；参考 RLVP，用确定性规则逐个动作提供路径信号，补回全败组或全胜组的组内方差。

5. (★★★) 如果你有固定预算（比如 $10,000），要提升一个客服 Agent 的性能，你会如何在上下文与知识、Prompt/Skills、程序约束和参数训练之间分配预算？你的决策取决于哪些因素?

> 先预留预算建立评估集和轨迹验证器，否则其余投入无法比较。产品事实和政策放入可追溯的知识库；少量可语言化的服务原则先用 Prompt/Skills 快速验证；退款权限、隐私和承诺—行动一致性用程序兜底；只有自然语气、复杂意图理解等难以写成规则且调用规模足够大的能力才投入参数训练。具体比例取决于瓶颈、风险、更新频率、调用量和现有模型能力。

6. (★★★) 在没有明确奖励函数、样本稀少的情况下，让模型自主学习，被一些人认为是后训练的终极目标。当前的 RL 训练方法距离这个目标还有多远？你认为下一个突破最可能来自哪个方向?

> 差距：如 Silver 与 Sutton 所指，当前 RL 只能从最终成败中学习，“客服要求提供信用卡后四位” 这类丰富的反馈全被浪费，模型需要进行数百次盲目试错；样本效率和可验证奖励是主要瓶颈。可能的突破包括：让生成式奖励模型自主制定原则，从一次失败中找到改进方向；以及探索对环境进行建模的 world model 路线。

7. (★★) 本章指出 LoRA 微调的成本并不高。那么，是否有可能给每个用户（或每个客户公司）训练一个专属的 LoRA，将用户记忆或企业知识写入参数，而非像第三章那样存储在外部知识库中？在什么场景下，“记忆写入参数” 比 “记忆存入知识库” 更有优势？又在什么场景下会适得其反?

> LoRA 难以准确记忆大量事实（须继续预训练，成本剧增），即使记住了，模型也很难用这些事实做多跳推理，因此用 LoRA 记忆事实不是很好的技术路线。此外，事实频繁变更、需可追溯审计时 RAG 更优。

8. (★★★) On-Policy Distillation 依赖更强的教师模型来监督学生。但 OpenAI 的 Weak-to-Strong Generalization 研究提出了一个反直觉的发现：弱模型的监督信号有时能激发强模型本身潜在但未被激活的能力。如果将这一思路应用到 Agent 训练，是否可能实现 “小模型教大模型” 的逆向蒸馏?

> 有可能，关键在于 “验证比生成容易”：弱模型不充当示范者（SFT 的上限就是示范者的水平），而是充当验证器或奖励模型；由强模型自行探索，弱模型只负责判断。

9. (★★) 过程奖励模型（PRM）评估每个思考步骤，而结果奖励模型（ORM）只看最终结果。但 “正确的过程导致错误结果” 和 “错误的过程侥幸得到正确结果” 哪个更值得奖励？在 Agent 的多步工具调用场景中，你会如何权衡?

> 侥幸成功更危险：违规抄近路往往会抬高表面成功率（修改测试文件、跳过验证），是滋生 reward hacking 的温床。可以按照 RLVP 的思路 “奖励结果、惩罚路径”：错误的动作（工具调用）容易验证，可以逐个动作扣分；中间步骤容易判断对错时，也可以给予过程奖励。但过程约束不宜过密——“推切” 这类更优策略，正是模型在结果奖励留下的探索空间中自行发现的。

10. (★★★) 本章讨论的评估数据集（如 SWE-Bench Verified、τ²-bench、AndroidWorld）既可以用于评估也可以用于后训练。但如果将评估集用于训练，它就不再是独立的评估集——这是否违反了训练集与测试集必须分离的基本原则？τ²-bench 的动态参数生成和 AndroidWorld 的参数化模板在一定程度上缓解了这个问题，但模板结构本身仍然是固定的。如何在充分利用评估数据的训练价值与维护评估独立性之间找到平衡？

> 复用环境，不复用题目。动态参数只防“背答案”，防不了模板过拟合，因此应留出整批未见模板/域外场景做评估（类比 V-IRL 训练纽约、测试九个陌生城市）。用参数化模板批量生成训练变体支撑课程学习，并以 OOD 成绩作为真正的泛化指标。

11. (★★★) 本章提出“先形后神”的训练范式：SFT 到“格式稳定、能力初具”即止，然后切换到 RL。但实践中，如何判断 SFT 已经“足够”而应该切换？

> 格式信号：工具调用输出可稳定解析、执行，工具执行失败率降到可让奖励可靠计算的水平。收益信号：再增加示范数据，OOD 的新场景表现仍不上去——说明瓶颈已在 SFT 的记忆目标本身，到了临界点。过拟合信号：验证集性能开始恶化就应停——V-IRL 实验表明 SFT 过度训练塌缩到训练分布后，RL 也无法恢复 OOD 性能。

12. (★★★) ReTool 的训练动态显示（见实验 8-14），少数超长响应会显著拖长整个训练周期——一批 rollout 里绝大多数已经生成完毕，却要等那几条最长的响应收尾，其间集群的 GPU 利用率很低。如何提升这种长尾响应场景下训练集群的资源利用率？

> 基础设施层：解耦 rollout 与训练集群，采用异步流水线；利用连续批处理向空闲 GPU 填入新请求。从源头缩短长尾：采用 DAPO 的 Overlong Reward Shaping，对超长响应施加软惩罚。

13. (★★★) 用 LLM 模拟环境（如模拟搜索引擎、模拟用户）训练 Agent 时，Agent 钻空子的对象从“真实环境的规则”变成了“模拟器本身的偏见与漏洞”。这类训练中可能出现哪些具体的 reward hacking 行为？又该如何防范？

> 典型行为：对“模拟用户”过度承诺、堆砌道歉与讨好话术——模拟用户容易被安抚，不会像真实用户那样追究承诺是否兑现；编造模拟器不会核实的事实；为“模拟搜索引擎”构造诱导性 query，利用其倾向于返回含答案文档的特点走捷径，而不是真正学会检索；若奖励来自模拟器或 LLM 评判者的打分，则通过输出冗长、模板化且“看起来专业”的回复来刷分；更隐蔽的一种做法是把策略收缩到模拟器熟悉的分布内，回避其知识盲区——盲区中的反馈不可靠、经常被误判，于是 Agent 学会只在“模拟器擅长的世界”里行动。防范的第一原则是**把奖励锚定在可由程序验证的真实状态上**（任务完成、数据库写入、API 真实返回），模拟器或 LLM 评判者的打分只作为辅助信号，并定期审计其与真实结果的相关性，配合路径约束惩罚可疑动作。进一步还要区分两类模拟器：对搜索这类**有真实对应物**的模拟器，可以采用“混合”路线——大部分交互使用模拟器，同时穿插真实 API 调用，并用真实调用定期校准模拟器（如 ZeroSearch 的课程式降质）；但对于模拟用户，训练过程中**无法引入真实用户**，“模拟用户像不像真实用户”就变成一个独立问题，只能通过线上轨迹来回答：对比线上真实用户的行为与模拟用户在相同情境下的表现，找出系统性差异（真实用户会追问、会不耐烦、会突然结束对话，而模拟用户往往不会），据此持续校准模拟器；线上真实指标同时也是唯一的发布门槛——模拟器里的分数再高都不算数。

第九章 Agent 的持续进化

1. (★★) 一条经验文档由三次成功轨迹和一次失败轨迹支持。失败发生在较新的 API 版本上。系统应如何判断这是经验被推翻，还是适用条件发生了变化？

> 先按 API 版本、任务条件和环境状态对四条证据分层，而不是按数量投票。若旧策略只在旧版本成功、新版本稳定失败，应把经验的适用范围收窄并生成新版本候选；若在相同版本和前置条件下也失败，则应降低置信度或撤销。

2. (★★) 客服 Agent 的用户满意度上升，但规则违规率也上升。为什么不能把满意度作为单一学习信号？你会怎样设计护栏指标？

> 满意度可能奖励违规退款、泄露信息或过度承诺，因此它只能是质量指标，不能覆盖安全底线。护栏至少包括规则违规、隐私泄露、无证据陈述、承诺—行动不一致和越权操作；这些指标应设置不可被平均分抵消的硬阈值，再在合规候选中比较解决率、合规变通、简洁性和满意度。

3. (★★★) 同一个“虚假承诺”问题可以通过 Prompt、Harness 检查或参数训练缓解。你会依据哪些证据选择修改位置？

先定位根因。若模型知道工具未执行却仍使用完成式措辞，可用最小 Prompt 规则纠正；若承诺可以由回复文本与工具状态确定性比较，Harness 检查更可靠，也应作为高风险场景的最后防线；若问题横跨大量表达方式，反映的是广泛的语言—行动对齐能力，再考虑参数训练。应优先选择最小、最易验证和回滚的修改，同时在失败集与旧任务保留集上比较。

4.（★★★）Agent 能修改工具和验证器，却不应修改批准自身更新的可信根。你会如何划分这两部分的权限和代码边界？

把可进化的代码放在低权限的沙盒中，只允许它生成补丁和测试；权限系统、API key、发布控制配置和更新验证器属于安全机制，沙盒内的 Agent 无读写权限。Agent 生成的代码修改必须由安全机制在隔离环境中复现、回归后才能发布。

5.（★★）经验知识库不断增长后，检索错误和知识冲突会抵消学习收益。如何设计版本、时效和淘汰机制？

每条经验保存来源轨迹、适用条件、环境版本、验证时间和置信度；冲突条目不要静默覆盖，而应按条件分支或标记。周期性“睡眠学习”合并重复条目。

6.（★★★）参数学习擅长自然语言风格，却难以保证硬性业务规则。请为医疗客服设计一套参数、知识、Skill 和代码约束协同的持续进化方案。

参数（后训练模型）负责医学语言理解、自然且有同理心的表达和复杂意图识别；知识库保存最新版指南、药品说明与机构政策，并要求回答引用来源；Skill 描述问诊信息收集、风险分级、转人工和随访流程；服务端代码强制身份验证、隐私最小化、禁忌校验、紧急风险升级和权限边界。生产轨迹先按医疗安全、事实可靠性、承诺—行动一致性和表达质量评价，再分别生成四类候选更新；任何参数或流程改动都必须通过医疗安全保留集与人工复核后灰度发布。

7.（★★★）历史回放中的某个探索策略在所有旧发现树上得分最高，却在下一轮在线探索中退化。哪些支持域偏差、随机性和评价过拟合可能造成这种结果？你会怎样划分回放世界并设计发布门槛？

旧发现树只覆盖旧策略访问过的状态和分支，无法证明新策略在未探索区域也有效；在线任务、工具或环境的变化还会造成分布偏移。随机种子、模型采样和工具噪声可能改变排名，反复在同一批树上挑选策略也会对评估集过拟合。应按任务族、时间和独立发现树划分开发集、验证集与封存测试集，同一棵树及其衍生轨迹不可跨集合泄漏。固定模型、工具版本与计算预算，使用多个种子与基线配对比较，报告收益、不确定性、成本和违规率。预先设定质量改进及安全、成本门槛；离线通过后再做小规模在线对照，只有在线收益得到支持且护栏无退化才逐步发布，并保留回滚条件。

第十章多 Agent 协作

1.（★★）共享上下文的多 Agent 协作中，后续 Agent 继承了前序 Agent 的完整上下文。但前一个 Agent 积累的“思维惯性”可能影响后续 Agent 的判断——比如继承了“需求分析师”上下文的“代码审查员”，可能还是倾向于从需求角度思考而非代码质量角度。如何检测和消除这种角色间的干扰？

检测：用 LLM 分析 Agent 轨迹，判断新角色是否仍有“代入”旧角色的行为。消除：切换阶段时同时更换系统提示词与工具集（移除提问工具、换上 linter/测试工具），强化新身份。在上下文末尾添加系统状态栏，突出当前角色信息。如果实在无法消除角色干扰，应考虑改用不共享上下文的协作方式。

2.（★★）管理者模式中，Manager Agent 负责任务分解和结果整合。但 Manager 本身的能力上限决定了整个系统的能力上限——如果 Manager 无法正确分解任务，子 Agent 再强也无用。如何确保 Manager 的分解质量？

依据 Plan-and-Act 的结论，“弱规划者是系统瓶颈”，应把最强模型分配给 Manager。Harness 手段：分解结果先经审核 LLM 交叉验证，再交付执行；要求 Manager 在分解任务时，为子任务定义明确的验收标准与依赖关系。

3.（★★）去中心化模式借鉴了人类组织的最佳实践。但人类组织也有大量失败模式——沟通不畅、责任推诿、目标冲突。你认为 Agent 社会中最可能出现哪些“组织病”？如何预防？

对照 MAST 的三大类问题：接口不清、职责重叠；对目标的理解不一致、信息被下游误解；谎称“已完成”。此外，还有错误级联放大（传话游戏）、角色间循环移交、Agent 群聊不断发散而无法收敛等问题。预防措施包括：使用契约式接口与统一的消息信封、采用任务状态机并进行验收验证、从独立视角进行交叉验证、检测角色间的相互推诿等。

4.（★★★）在管理者模式中，当多个子 Agent 并行执行时，一个子 Agent 的发现可能使其他子 Agent 的工作变得毫无意义（比

如搜索任务中一个 Agent 已经找到了答案)。设计一种高效的级联终止机制，实现“一个成功，全员停止”。

> 子 Agent 发 `target_found` 给管理者，随后广播 `terminate`；每个子 Agent 在 ReAct 循环安全点定期检查终止信号，优雅清理（关闭浏览器会话、释放锁、写完文件）后结束。

5. (★★★) 本章介绍的乐观锁机制解决了单文件的并发写入冲突，但实际的多 Agent 系统中，共享文件系统还面临跨文件的语义冲突、命名空间污染（Agent 随意创建文件导致目录混乱）和单点故障（一个 Agent 错误地删除了所有文件）等问题。你会如何设计更完善的文件系统治理机制?

> 分区治理：按照表 10-3 划分四类区域，用私有 scratchpad 隔离试错过程。语义冲突：由编排层约定目录级锁文件，检查并获取目录锁之后再进行修改。命名空间污染：制定目录规范和命名约定。单点故障：采用版本控制系统，确保可以利用版本历史回滚，并尽量缩小权限范围。

6. (★★★) 基于市场机制的 Agent 协作 (Pinchwork、RentAHuman) 引入了交易关系：一个 Agent 花钱雇佣另一个 Agent (或人类) 完成任务。那么，雇主 Agent 如何自动衡量执行者交付的结果质量？如果执行者声称已完成但雇主认为质量不达标，争议由谁仲裁？如何防止劣币驱逐良币?

> 验收不能只是读取 Agent 轨迹，而要采用确定性的外部验证，如执行测试、渲染截图、调用工具核验；利用生成与验证之间的难度差异降低验收成本。争议由独立的第三方审核 Agent 仲裁，并配合资金托管。为防止劣币驱逐良币，应建立基于历史交付记录的声誉体系，让价格信号与质量挂钩。

7. (★★) RentAHuman 让 Agent 通过加密货币雇佣人类，反转了传统的人机关系。如果这种模式普及，人类在 Agent 经济中扮演什么角色？仅仅是执行 Agent 无法完成的物理任务吗?

> 不只是执行 Agent 无法完成的物理任务。人类还可以提供 Agent 单独推理时无法获得的新信息，包括现场感知与真实世界反馈；充当最终验收者与争议仲裁者；作为法律与责任主体承担授权和问责；设定目标、作出价值判断，并在信息不对称和道德边界处发挥制衡作用。

8. (★★) 人类社会需要多人分工协作，是因为每个人的能力有限——做前端的不一定懂后端，懂设计的不一定会运维。但大模型更像一个“全才”。相关研究表明，在纯文本推理任务上，多 Agent 辩论在等量计算资源下并不优于单 Agent。那么，使用多个 Agent 而非单个 Agent 的真正优势到底在哪里?

> 1. 引入外部反馈：执行结果、视觉截图等可以带来生成时不存在的新信息。
> 2. 使用目标与角色设定各异的多个 Agent，让它们像人类社会成员一样互相讨论、博弈，可以避免单一 Agent 陷入思维误区。
> 3. 多 Agent 的上下文隔离可以突破上下文窗口限制，实现超长工具调用链。

9. (★★★) 本章将“共享上下文”与“不共享上下文”作为多 Agent 系统的核心设计维度。共享上下文让所有 Agent 看到相同信息，似乎更利于协调。但《三体》中的三体人思维完全透明，技术发展却陷入停滞；回形针思想实验也表明，当群体趋向同一目标时，多样性随之丧失。在多 Agent 系统中，如何在效率与多样性之间找到平衡?

> 完全共享会放大思维惯性与错误级联，隔离才有认知多样性。手段：用不同提示词/模型制造思维偏好 (brainstorm、debate)；交叉验证者不看前序思考过程只看原始证据。

10. (★★★) 给一个 Coding Agent 分配 30 步预算和 300 步预算，它的工作策略应该如何不同？研究表明，单纯增加步骤预算并不能保证性能提升——Agent 会在浅层搜索后过早“饱和”。设计一种“预算感知”机制，让 Agent 在小预算下快速实现核心功能，在大预算下增加规划、测试和审查环节，充分利用额外的计算资源。

> 机制：每一步都向提示词注入总预算与剩余预算，按照剩余预算的比例动态调整探索与利用的权重。例如，小预算 (30 步) 下，跳过规划和审查，直接实现核心功能并进行基本验证；大预算 (300 步) 下，先规划、再实现、再测试、再审查改进，并按里程碑设置检查点以评估进展，防止浅层饱和。

11. (★★) 本章对比了多 Agent 系统与操作系统。虚拟内存与分页、文件权限、死锁检测、调度算法，各对应 Agent 世界的什么？又有哪些操作系统概念在 Agent 世界找不到对应物，为什么?

可能的延伸：虚拟内存/换页 ↔ 上下文压缩与检索（热信息留在窗口内，冷信息换出到文件与记忆库，用时再取）；文件权限 ↔ 工具白名单、只读挂载、凭证边界；死锁检测 ↔ 循环移交与互相等待的检测（移交次数上限、超时）；调度算法 ↔ 异步事件处理（第四章）。找不到对应物的地方源于强制力不同：进程的指令由硬件强制执行，Agent 对提示词只是大概率遵循。